

Planning

Lara J. Martin (she/they)

<https://laramartin.net/neurosymbolic-text-gen/>

Slides adapted from Chris Callison-Burch

Learning Objectives

Identify the components of a planning problem

Distinguish between search and planning

Discover limitations and benefits of planning for storytelling & games

Explore how Theory of Mind can expand narrative planning in the Sabre planner

Review: AI Agent Definition

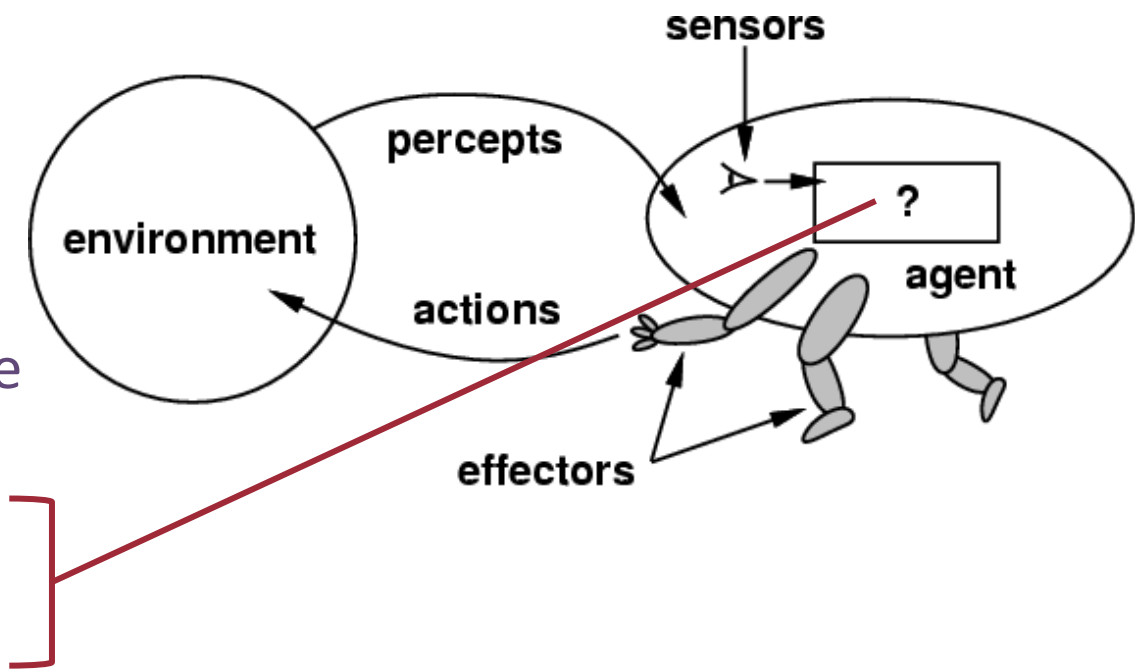
Agent: anything that **perceives** its environment through **sensors**, and **acts** on its environment through **actuators**

Percept: input at an instant

Percept sequence: history of inputs

Agent function: mapping of **percept sequence** to **action**

Agent program: (concise) implementation of an agent function



Picture from Dr. Cynthia Matuszek, source unknown

Review: What are the components of a search problem?

Break down the problem space into **states** and actions (for each state)

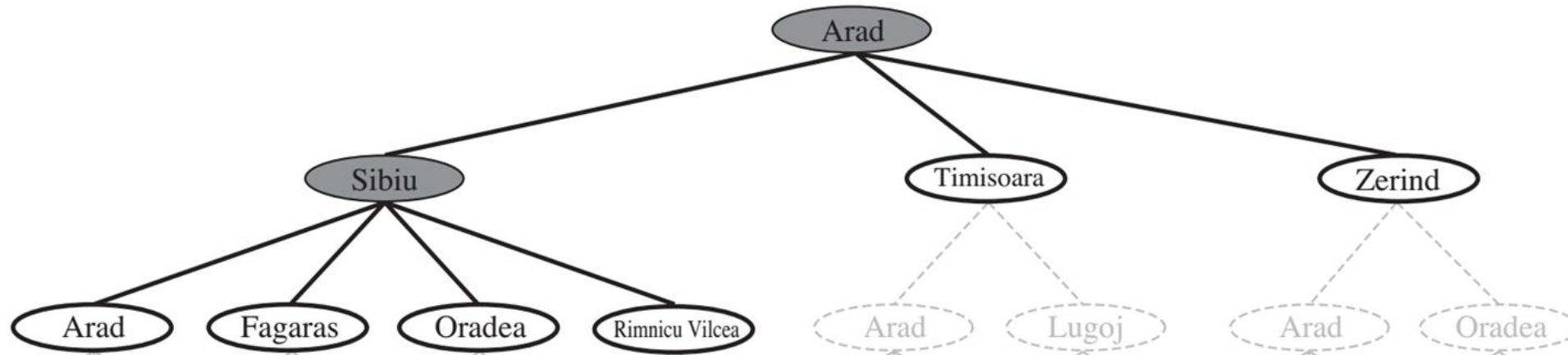
+ these components:

- Initial state – where the agent starts
- Transition model – how actions change states
- Goal test – measuring if the current state is the goal
- Step cost (optional) – how expensive it is to take action a in state s

Problem: Find a sequence of **actions** that **transitions** the **initial state** into a state which passes the **goal test**

Solution: sequence of actions, or a **plan**

Review: Generalized tree search



function TREE-SEARCH(*problem*, *strategy*) return a solution or failure

Initialize frontier to the *initial state* of the *problem*
do

if the frontier is empty then return *failure*

choose leaf node for expansion according to strategy & remove from frontier

if node contains goal state then return *solution*

else expand the node and add resulting nodes to the frontier

The strategy determines search process!

Review: Search Strategies

Several classic search algorithms differ only by the order of how they expand their search trees

You can implement them by using different queue data structures

- **Depth-first search** = LIFO queue
- **Breadth-first search** = FIFO queue
- **Greedy best-first search** or **A* search** = Priority queue

Planning

Planning: The process of **searching** for a plan

- This is why we can use algorithms like BFS to find plans
- “Plain” state-based search is useful when we just want to get to the goal (efficiently); planning is useful when we care about the *path*
- What we think of as “planning” is a combination of search and logic

Plan: The result of planning; a sequence of steps from the initial state to a goal state

Policy: A collection of transition functions ($\text{Result}(s, a) \rightarrow s_r$) that tell the agent what action it should take for a given state

- This will become more relevant when talking about reinforcement learning

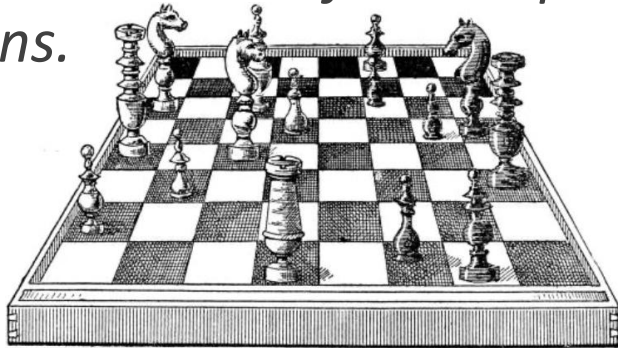
Classical Planning

The task of finding a sequence of action to accomplish a goal in a deterministic, fully observable, discrete, static environment.

If an environment is:

- **Deterministic**
- **Fully observable**

The solution to any problem in such an environment is a fixed sequence of actions.



In environments that are

- **Nondeterministic** or
- **Partially observable**

The solution must recommend different future actions depending on the what percepts it receives. This could be in the form of a *branching strategy*.



More complicated planning

In environments that are

- Nondeterministic
- Partially observable
- Etc.

—————→ Nondeterministic actions
(with assigned probabilities)
turn classical planning
problems into a Markov
Decision Process (MDP)!

The solution must recommend different future actions depending on the what percepts it receives. This could be in the form of a branching strategy.



Planning languages

PDDL (Planning Domain Definition Language)

STRIPS (Stanford Research Institute Problem Solver)

ADL (Action Description Language)

...

We'll focus on PDDL

PDDL breaks the planning problem into a **domain** and a **problem description**

The **domain** is consistent across problems (e.g., the description of the environment)

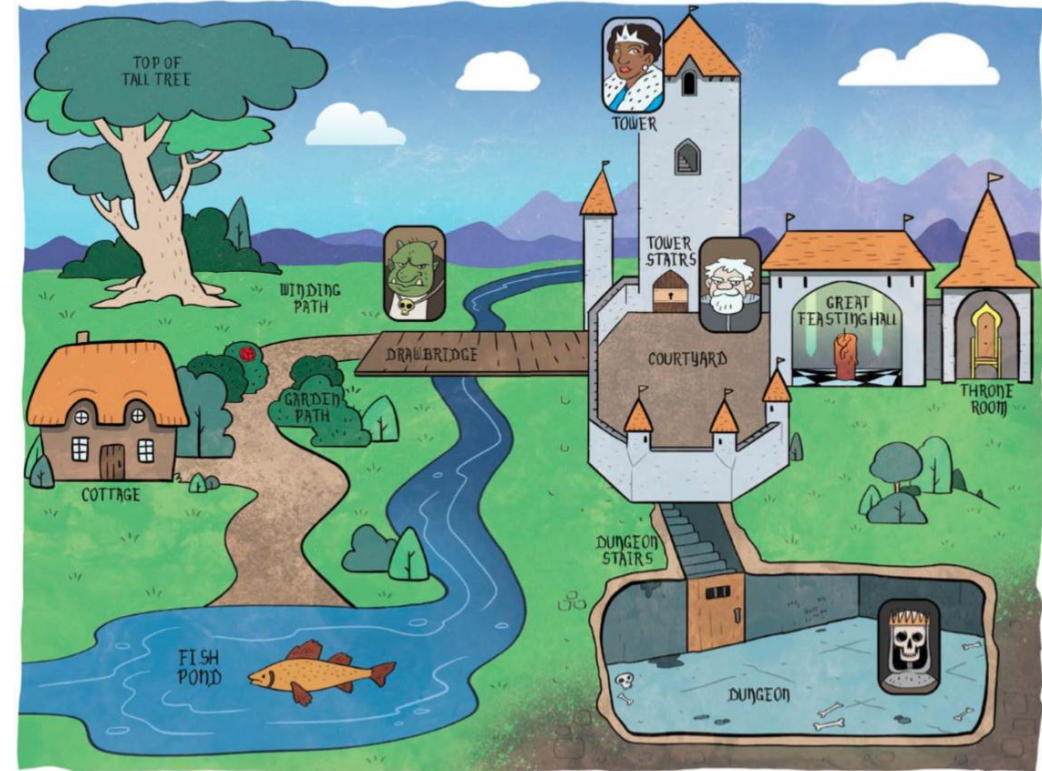
The **problem** defines what is going to be planned over

Example: Action Castle

We want to create a domain & problem that will play an interactive fiction game.

The resulting plan will be a sequence of actions that the user can take to complete the game.

- | | | |
|-----------------------------|---------------------------|-------------------------------|
| 1. take pole | 13. go east | |
| 2. go out | 14. hit guard with branch | |
| 3. go south | 15. get key | |
| 4. catch fish with pole | 16. go east | |
| 5. go north | 17. get candle | |
| 6. pick rose | 18. go west | |
| 7. go north | 19. go down | |
| 8. go up | 20. light lamp | 26. go up |
| 9. get branch | 21. go down | 27. go up |
| 10. go down | 22. light candle | 28. unlock door |
| 11. go east | 23. read runes | 29. go up |
| 12. give the troll the fish | 24. get crown | 30. give rose to the princess |
| | 25. go up | 31. propose to the princess |
| | | 32. down |
| | | 33. down |
| | | 34. east |
| | | 35. east |
| | | 36. wear crown |
| | | 37. sit on throne |



Domain

```
(define (domain action-castle)
  (:requirements :strips :typing)
  (:types
```

Domain name

```
    player location direction
    fish crown - item
  )
```

Object Types (can be hierarchical)

```
(:action go
```

Parameters
(variables)

```
  :parameters (?dir - direction ?p - player ?l1 - location ?l2 - location)
```

Preconditions

```
  :precondition (and (at ?p ?l1)
                     (connected ?l1 ?dir ?l2)
                     (not (blocked ?l1 ?dir ?l2)))
```

Effects

```
  :effect (and (at ?p ?l2)
               (not (at ?p ?l1)))
```

Logical statements

```
)
```

```
(:action get
```

```
  :parameters (?item - item ?p - player ?l1 - location )
```

```
  :precondition (and (at ?p ?l1)
                     (at ?item ?l1))
```

```
  :effect (and (inventory ?p ?item)
               (not (at ?item ?l1)))
```

```
))
```

Actions

Problem

Objects
(the atoms)

```
(define (problem navigate-to-location)
  (:domain action-castle)

  (:objects
    npc - player
    cottage gardenpath fishingpond gardenpath
    windingpath talltree drawbridge courtyard
    towerstairs tower dungeonstairs dungeon
    greatfeastinghall throneroom - location
    in out north south east west up down - direction
  )

  (:init
    (at npc cottage)
    (connected cottage out gardenpath)
    (connected gardenpath in cottage)
    (connected gardenpath south fishingpond)
    (connected fishingpond north gardenpath)
  )

  (:goal (and (at npc throneroom) (sitting npc throne)))
)
```

Problem name

What domain to
use

Initial State

Goal

State Representation

In PDDL, a **state** is represented as a **conjunction** of **logical sentences** that are **ground atomic fluents**.

Ground means
they contain no
variables

Atomic sentences:
logical statements
that can't be
broken down

Fluent means an aspect
of the world that can
change over time

E.g.,

(*connected* gardenpath in cottage)

We make the *closed world assumption*: any fluent not mentioned is false

Algorithms for Classical Planning

We can apply **BFS** to the **initial state** through possible states looking for a **goal**.

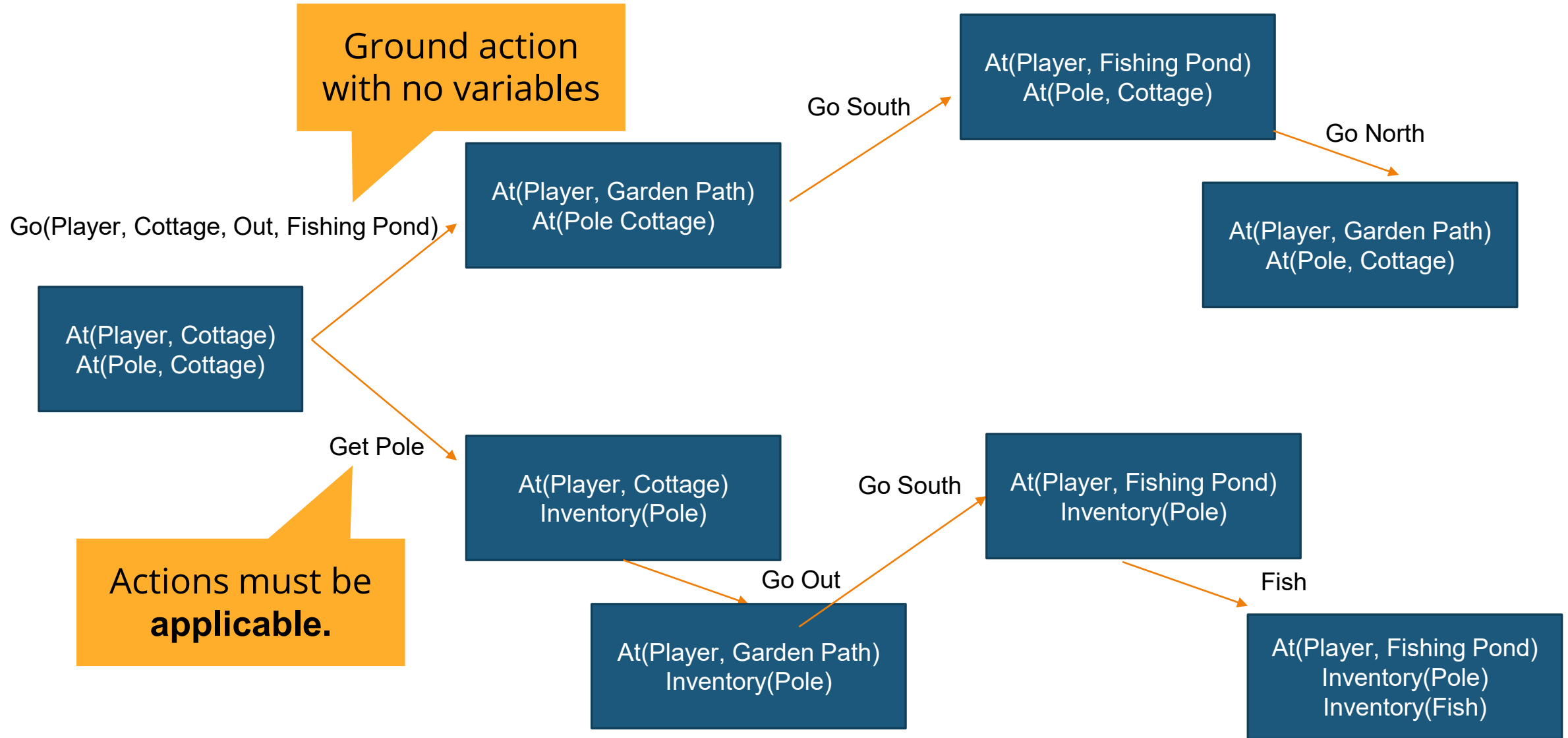
An advantage of the **declarative representation** of action schemas is that we can also **search backwards**.

Start with a goal and work backwards towards the initial state.

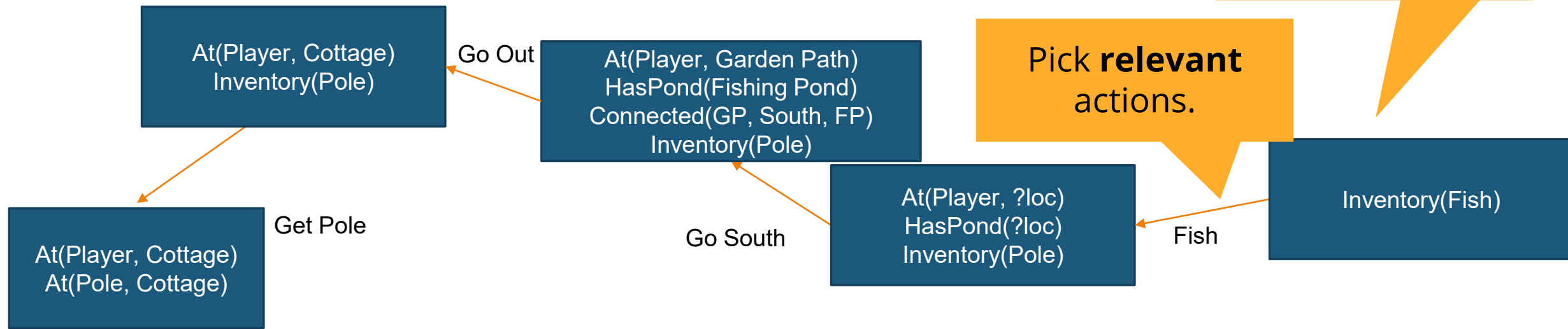
Why work
backwards?

In our Action Castle example, this would help us with the branching problem that the `drop` action introduced. If we work backwards from the goal, then we realize that we don't ever need to drop an item for the correct solution.

Forward State-Space Search for Planning



Backward State-Space Search for Planning (aka Regression Search)



Given a goal **g** and action **a**, the **regression** from g to a gives a state **g'** description whose literals are given by:

$$\text{POS}(g') = (\text{POS}(g) - \text{ADD}(a)) \cup \text{POS}(\text{Preconditions}(a))$$
$$\text{NEG}(g') = (\text{NEG}(g) - \text{DEL}(a)) \cup \text{NEG}(\text{Preconditions}(a))$$

Negative literals in the effects are kept in a **delete list DEL**

Positive literals in the effects are kept in an ADD list

Backward State-Space Search for Planning (aka Regression Search)

Given a goal **g** and action **a**, the **regression** from g to a gives a state **g'** description whose literals are given by:

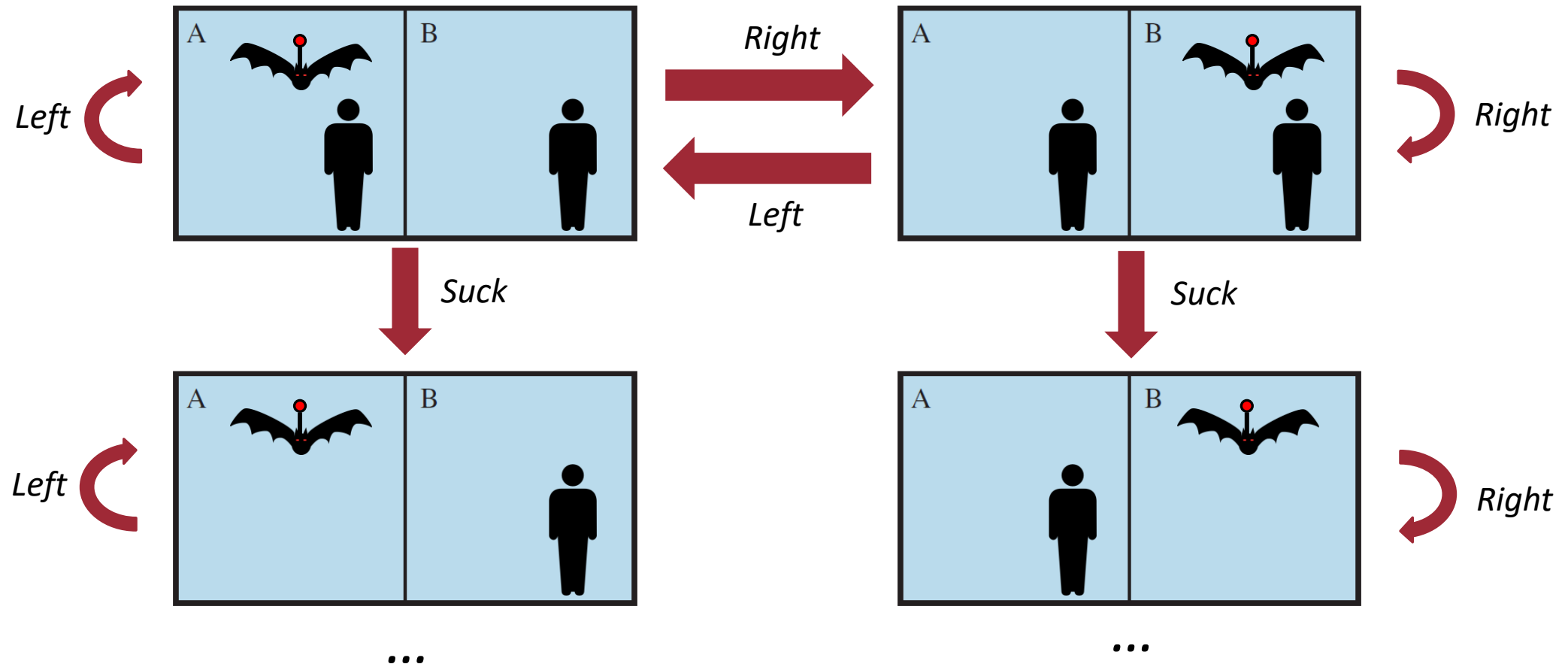
$$\text{POS}(g') = (\text{POS}(g) - \text{ADD}(a)) \cup \text{POS}(\text{Preconditions}(a))$$

$$\text{NEG}(g') = (\text{NEG}(g) - \text{DEL}(a)) \cup \text{NEG}(\text{Preconditions}(a))$$

Or simply:

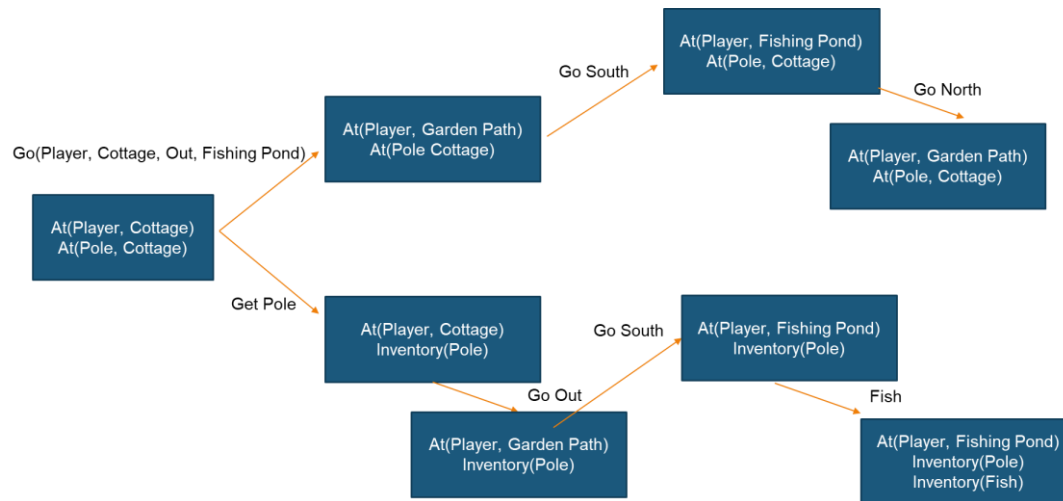
$$g' = (g - \text{effects}(a)) \cup \text{Preconditions}(a)$$

Search Review: Vampire World: Transition Model Example

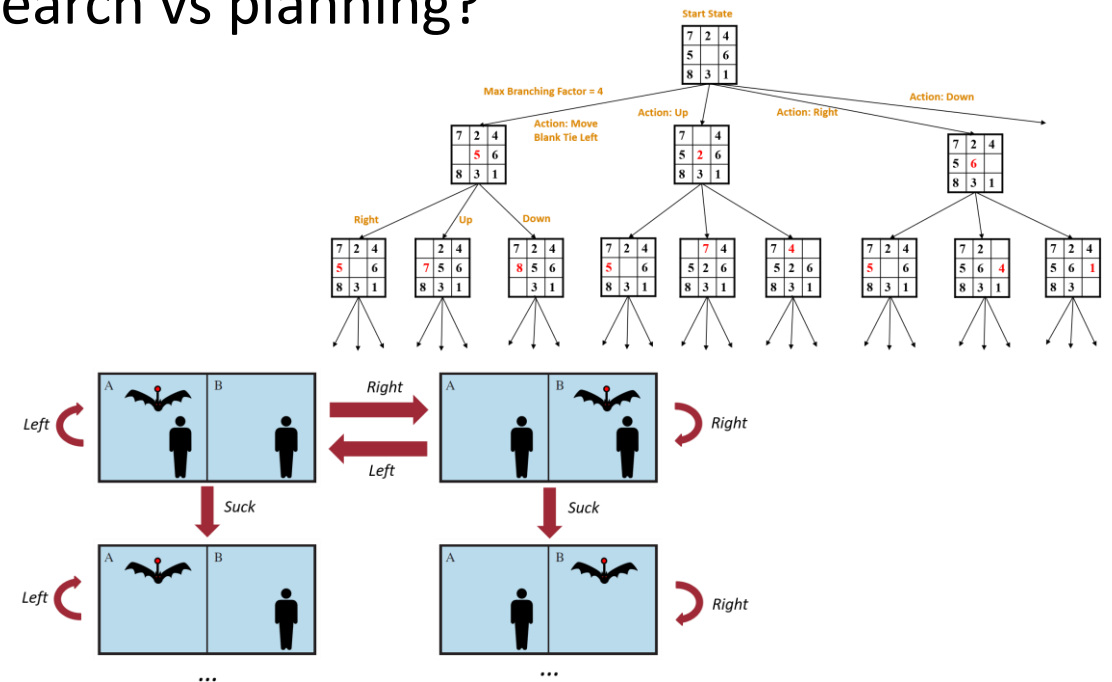


Think-Pair-Share: Search vs Planning

What are some of the differences between search vs planning?



Planning



Search

Heuristics for Planning

Neither forward nor backward search is efficient without good **heuristics**.

In search, a heuristic function $h(s)$ estimates the distance from a state to the goal.

Admissible heuristics never over-estimate the true distance and can be used with **A* search** to find optimal solutions.

Admissible heuristics can be derived from a **relaxed problem** (approximation) that is easier to solve.

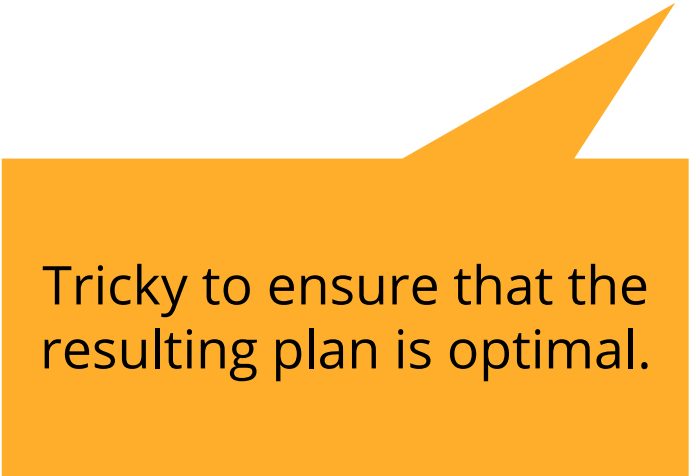
The “**ignore preconditions**” heuristic relaxes the problem.

Hierarchical Planning

Instead of using atomic actions, we can define actions at **higher levels of abstraction**.

Hierarchical decomposition organizes actions into high-level functions, composed of more fine-grained function, composed of atomic actions.

Plan out sequence of high-level actions, recursively **refine the plan** until we've got atomic actions.



Tricky to ensure that the resulting plan is optimal.

Planning and Games

Planning can be used for AI characters

In our current text adventure games, all of the non-player characters are boring!

Why doesn't the princess try to escape the tower and claim the throne herself?

Why doesn't the troll go hunt for food and eat us or the guard?

Why is the ghost of the king stuck in the dungeon?

We could give each of them goals and have them try to plan out and play the game alongside the player.

(Teaser for
HW 3)

Generating Puzzles

In HW1, we were able to generate descriptions of locations and items.

Could we use planning to automatically generate:

1. Puzzles?
2. Special actions?

Let's say a player needs a **sword** and we decide to make the game more challenging by not putting one anywhere in the game.

Could we generate an action that results in the creation of a sword?

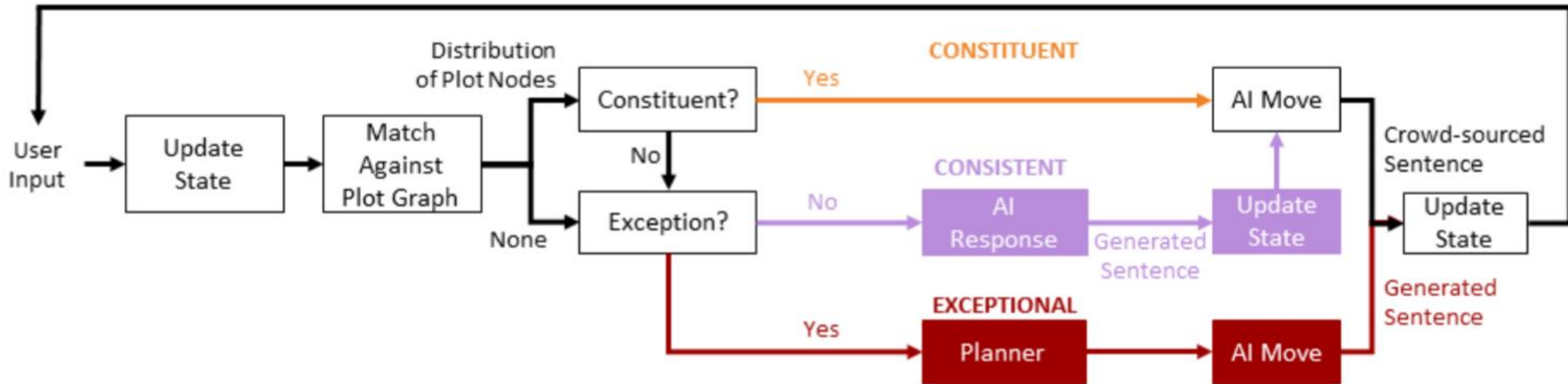
Action: forge a sword

Effects: a sword is created

Preconditions: molten metal, a cast of a sword, an anvil, a hammer

Planning and Stories

Planning for IF



UNIVERSE

Table 2
A typical UNIVERSE plot fragment.

PLOT FRAGMENT: forced-marriage
CHARACTERS: ?him ?her ?husband ?parent
CONSTRAINTS: (has-husband ?her) {the husband character}
(has-parent ?husband) {the parent character}
(< (trait-value ?parent 'niceness) – 5)
(female-adult ?her)
(male-adult ?him)
GOALS: (churn ?him ?her) {prevent them from being happy}
SUBGOALS: (do-threaten ?parent ?her “forget it”) {threaten ?her}
(dump-lover ?her ?him) {have ?her dump ?him}
(worry-about ?him) {have someone worry about ?him}
(together * ?him) {get ?him involved with someone else}
(eliminate ?parent) {get rid of ?parent (breaking threat)}
(do-divorce ?husband ?her) {end the unhappy marriage}
(or (churn ?him ?her) {either keep churning or}
(together ?her ?him)) {try and get ?her and ?him back together}

UNIVERSE (with multiple goals)

```
*(tell '(((churn JOSHUA FRAN)) ((together JOSHUA VALERIE))))  
  
working on goal -- CHURN JOSHUA FRAN  
  -- using plan ACCIDENT-BREAKUP P1/FRAN P2/JOSHUA THIRD-PARTY/VALERIE  
  
working on goal -- DO-DISABLE FRAN  
  -- using plan DISABLE PERSON/FRAN  
  
>>> FRAN has a spinal injury and is paralyzed  
  
>>> FRAN doesn't want to ruin JOSHUA's life  
  
>>> FRAN pretends to blame JOSHUA for her malady  
  
working on goal -- DUMP-LOVER FRAN JOSHUA  
  -- using plan BREAK-UP DUMPER/FRAN DUMPED/JOSHUA  
  
>>> FRAN tells JOSHUA she doesn't love him  
  
working on goal -- TOGETHER JOSHUA VALERIE  
  
[again, the story continues unhappily for almost all concerned]
```

Figure 3: A multi-goal story

Partial-Order Planning

Keep a **partial order of steps** and only commit to an ordering when forced to

For example:

- Go north
- Pick up sword; Pick up lantern
- Go west

Partial Order Causal Link (POCL) planning

Conflict POCL

Figure 1: Example CPOCL Problem and Domain

Initial: $\text{single}(A) \wedge \text{single}(B) \wedge \text{single}(C) \wedge$
 $\text{loves}(A, C) \wedge \text{intends}(A, \text{married}(A, C)) \wedge$
 $\text{loves}(B, C) \wedge \text{intends}(B, \text{married}(B, C)) \wedge \text{has}(B, R)$

Goal: $\text{married}(A, C)$

lose(?p, ?i)
 A: $\emptyset$
 P: $\text{has}(\text{?p}, \text{?i})$
 E: $\text{lost}(\text{?i}) \wedge \neg \text{has}(\text{?p}, \text{?i})$

give(?p1, ?p2, ?i)
 A: ?p1 ?p2
 P: $\text{has}(\text{?p1}, \text{?i})$
 E: $\text{has}(\text{?p2}, \text{?i}) \wedge \neg \text{has}(\text{?p1}, \text{?i})$

propose(?b, ?g)
 A: ?b
 P: $\text{loves}(\text{?b}, \text{?g}) \wedge \text{has}(\text{?b}, R)$
 E: $\text{loves}(\text{?g}, \text{?b}) \wedge \text{intends}(\text{?g}, \text{married}(\text{?b}, \text{?g}))$

find(?p, ?i)
 A: $\emptyset$
 P: $\text{lost}(\text{?i})$
 E: $\text{has}(\text{?p}, \text{?i}) \wedge \neg \text{lost}(\text{?i})$

marry(?b, ?g)
 A: ?b ?g
 P: $\text{loves}(\text{?b}, \text{?g}) \wedge \text{loves}(\text{?g}, \text{?b})$
 $\wedge \text{single}(\text{?b}) \wedge \text{single}(\text{?g})$
 E: $\text{married}(\text{?b}, \text{?g}) \wedge$
 $\neg \text{single}(\text{?b}) \wedge \neg \text{single}(\text{?g})$

Figure 2: Example CPOCL Plan

